
VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF ELECTRICAL AND ELECTRONIC ENGINEERING



LAB REPORT

SUBJECT: INTRO TO IC DESIGN (EE3201)

LAB 1: RTL DESIGN

Class: L02

Semester: 251

Lecturer: Mcs. Nguyen Trung Hieu

Student who has done the report:

Group: 13

Student name	ID number	Contribution
Nguyễn Đức Tiến	2313432	100%
Châu Thành Tài	2312988	100%
Võ Phan Tiến Đạt	2310716	100%

Ho Chi Minh City - 2025

LAB 1: RTL DESIGN

THÍ NGHIỆM 1

Mục tiêu:

Hiểu cách lập trình một hệ thống để cộng giá trị của đầu vào A vào chính nó lặp lại nhiều lần (mạch cộng dồn – accumulator).

Mô tả:

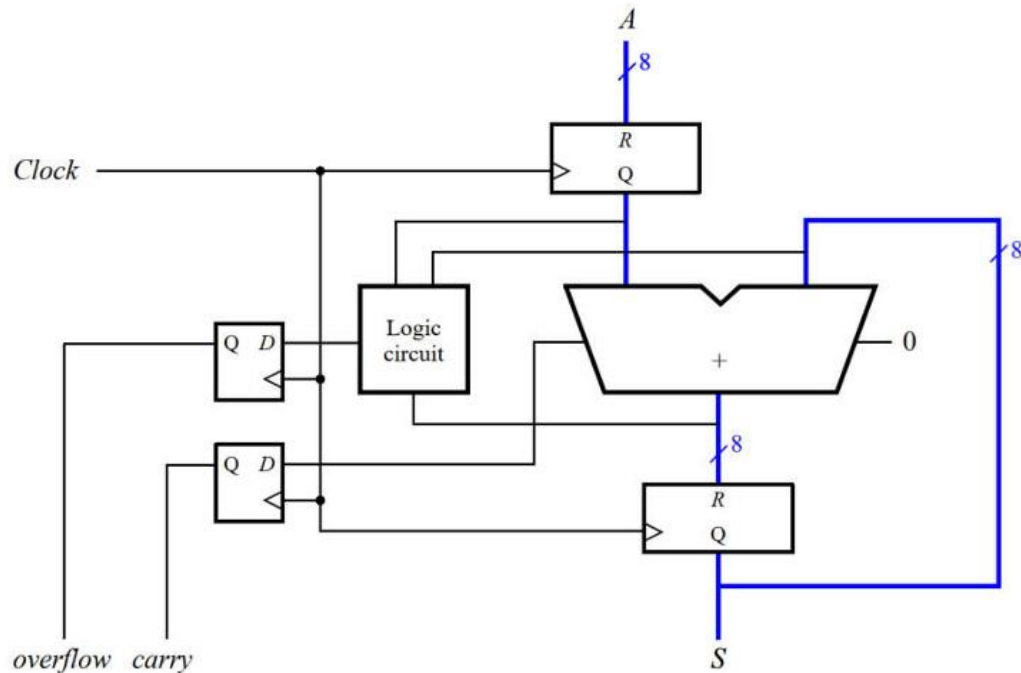
Mạch gồm bộ cộng 8-bit có ngõ ra carry out và overflow. Nếu A được xem là số bù 2 thì tín hiệu overflow = 1 khi kết quả bị tràn trong biểu diễn bù 2. Phát hiện tràn (overflow) Có 2 cách:

So sánh dấu của đầu vào và đầu ra: nếu khác dấu → tràn.

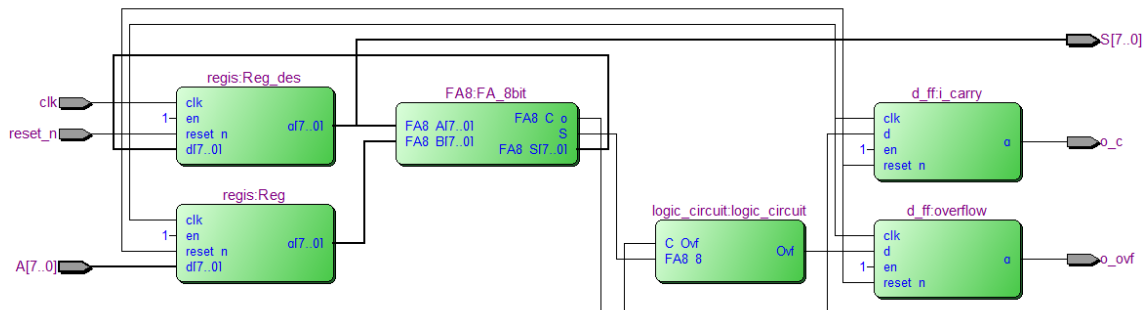
So sánh các bit carry: dùng mạch kiểm tra cờ tràn 8-bit.

Yêu cầu:

Dạng sóng (waveform) chứng minh mạch hoạt động đúng, thể hiện rõ các trường hợp overflow và carry.



Vẽ sơ đồ mạch:



Ý tưởng code:

Mạch sẽ bao gồm các bộ là bộ cộng 8 bit, 2 D_Flip Flop, 2 Regfile và bộ logic circuit để xét cờ tràn có dấu. Bộ cộng 8 bit sẽ được tạo thành từ 8 bộ full adder 1 bit, ban đầu nhập giá trị lưu vào thanh ghi A, giá trị sau khi tính sẽ được lưu vào thanh ghi S khi xung clock ra 1 và đồng thời bộ logic design sẽ kiểm tra nếu phép tính có bị tràn có dấu không để xuất ra overflow.

Code systemverilog:

- Bộ D_Flip Flop 1 bit

```
module d_ff (  
    input logic clk,  
    input logic reset_n,  
    input logic en,  
    input logic d,  
    output logic q  
);  
always_ff @(posedge clk or negedge reset_n) begin  
    if (!reset_n) q <= 1'b0;  
    else if (en) q <= d;  
end  
endmodule
```

- Bộ thanh ghi lưu giá trị S và A

```
module regis (
```

```

input logic    clk,
input logic    reset_n,
input logic    en,
input logic [7:0] d,
output logic [7:0] q
);
d_ff b0 (clk, reset_n, en, d[0], q[0]);
d_ff b1 (clk, reset_n, en, d[1], q[1]);
d_ff b2 (clk, reset_n, en, d[2], q[2]);
d_ff b3 (clk, reset_n, en, d[3], q[3]);
d_ff b4 (clk, reset_n, en, d[4], q[4]);
d_ff b5 (clk, reset_n, en, d[5], q[5]);
d_ff b6 (clk, reset_n, en, d[6], q[6]);
d_ff b7 (clk, reset_n, en, d[7], q[7]);
endmodule

```

- Logic circuit

```

module logic_circuit(
    input C_Ovf, FA8_8,
    output logic Ovf
);
assign Ovf = C_Ovf ^ FA8_8;
endmodule

```

- Bộ full_adder 1 bit

```

module fa(
    input logic FA_a, FA_b, C_i,
    output logic FA_S, C_o
);
assign FA_S = FA_a ^ FA_b ^ C_i;
assign C_o = (FA_a & FA_b) | (FA_b & C_i) | (FA_a & C_i);
endmodule

```

- Bộ cộng full adder 8 bit

```

module FA8 (
    input logic [7:0] FA8_A, FA8_B,
    output logic [7:0] FA8_S,
    output logic      FA8_C_o, S
);
wire [7:0] Bx = FA8_B ^ {8{1'b0}};

```

```

wire [8:0] c;
assign c[0] = 1'b0;

fa fa0
(.FA_a(FA8_A[0 ]), .FA_b(Bx[0 ]), .C_i(c[0 ]), .FA_S(FA8_S[0 ]), .C_o(c[1 ]));
fa fa1
(.FA_a(FA8_A[1 ]), .FA_b(Bx[1 ]), .C_i(c[1 ]), .FA_S(FA8_S[1 ]), .C_o(c[2 ]));
fa fa2
(.FA_a(FA8_A[2 ]), .FA_b(Bx[2 ]), .C_i(c[2 ]), .FA_S(FA8_S[2 ]), .C_o(c[3 ]));
fa fa3
(.FA_a(FA8_A[3 ]), .FA_b(Bx[3 ]), .C_i(c[3 ]), .FA_S(FA8_S[3 ]), .C_o(c[4 ]));
fa fa4
(.FA_a(FA8_A[4 ]), .FA_b(Bx[4 ]), .C_i(c[4 ]), .FA_S(FA8_S[4 ]), .C_o(c[5 ]));
fa fa5
(.FA_a(FA8_A[5 ]), .FA_b(Bx[5 ]), .C_i(c[5 ]), .FA_S(FA8_S[5 ]), .C_o(c[6 ]));
fa fa6
(.FA_a(FA8_A[6 ]), .FA_b(Bx[6 ]), .C_i(c[6 ]), .FA_S(FA8_S[6 ]), .C_o(c[7 ]));
fa fa7 (.FA_a(FA8_A[7 ]), .FA_b(Bx[7 ]), .C_i(c[7 ]), .FA_S(FA8_S[7 ]), .C_o(c[8 ]));

assign FA8_C_o = c[8];
assign S = c[7];
endmodule

```

- Bộ hoàn chỉnh

```

module Bai1_lab1(
    input logic [7:0] A,
    input logic clk,
    input logic en,
    input logic reset_n,
    output logic o_ovf, o_c,
    output logic [7:0] S
);

logic [7:0] FA_o;
logic FA_co;
logic [7:0] i_reg;
logic i_clk;
logic i_rd;
logic ovf;
logic to_ovf_d;
assign i_clk = clk;

//====THANH GHI CHUA GIA TRI DAU VAO====

```

```

regis Reg(
    .d (A),
    .reset_n(reset_n),
    .en    (en),
    .q (i_reg),
    .clk (i_clk));

//==== FA8BIT =====
FA8 FA_8bit(
    .FA8_A (S),
    .FA8_B (i_reg),
    .FA8_S (FA_o),
    .FA8_C_o(FA_co),
    .S(ovf)
);

//====THANH GHI DICH = S +A=====
regis Reg_des
(
    .d (FA_o),
    .reset_n(reset_n),
    .en    (en),
    .q (S),
    .clk (i_clk));

//===== KHOI XU TIN HIEU DAU RA=====
logic_circuit logic_circuit(
    .C_Ovf (FA_co),
    .FA8_8 (ovf),
    .Ovf(to_ovf_d)
); //XU LI TIN HIEU

d_ff overflow(
    .d(to_ovf_d),
    .reset_n(reset_n),
    .en    (en),
    .q (o_ovf),
    .clk (i_clk)
); //OVERFLOW

d_ff i_carry(
    .d (FA_co),
    .reset_n(reset_n),

```

```

.en    (en),
.q (o_c),
.clk (i_clk)
);// CO CARRY
endmodule

```

Testbench:

```

module tb_Bai1_lab1;

    // --- Khai báo tín hiệu ---
    logic [7:0] A;
    logic      en;
    logic      clk;
    logic      reset_n;
    logic      o_ovf;
    logic      o_c;
    logic [7:0] S;

    // --- Instantiate DUT
    Bai1_lab1 dut (
        .A      (A),
        .en      (en), // Nối chân en
        .clk      (clk),
        .reset_n (reset_n),
        .o_ovf    (o_ovf),
        .o_c      (o_c),
        .S        (S)
    );

    // --- Tạo Clock ---
    initial begin
        clk = 0;
        forever #5 clk = ~clk; // Chu kỳ 10ns
    end

    // --- Lệnh để tạo file dạng sóng ---
    initial begin
        $dumpfile("tb_Bai1_lab1.vcd");
        $dumpvars(0, tb_Bai1_lab1);
    end

    // --- Kịch bản Test

```

```

initial begin
    // --- 1. Khởi tạo và Reset ---
    A      = 8'd0;
    en     = 1'b0; // Tắt enable trong lúc reset
    reset_n = 1'b0; // Kích hoạt Reset (mức thấp)
    #20;        // Giữ reset trong 20ns
    reset_n = 1'b1; // Nhả Reset
    en      = 1'b1; // Bật Enable lên và giữ nguyên

    @(posedge clk); #1; // Đợi 1 xung clk để S reset về 0

    // --- 2. Test phép cộng đơn giản (en = 1) ---
    A = 8'd10;
    @(posedge clk); #1; // S = 0 + 10 = 10

    A = 8'd20;
    @(posedge clk); #1; // S = 10 + 20 = 30

    // --- 3. Test CARRY (o_c = 1) (en = 1) ---
    // S đang là 30. Ta cộng thêm A = 250.
    // Phép toán: 30 + 250 = 280
    // S sẽ = 24 và cờ Carry (o_c) sẽ = 1
    A = 8'd250;
    @(posedge clk); #1; // S = 24, o_c = 1

    // --- 4. Test OVERFLOW (o_ovf = 1)
    // S đang là 24 (số dương). Ta cộng thêm A = 110 (số dương).

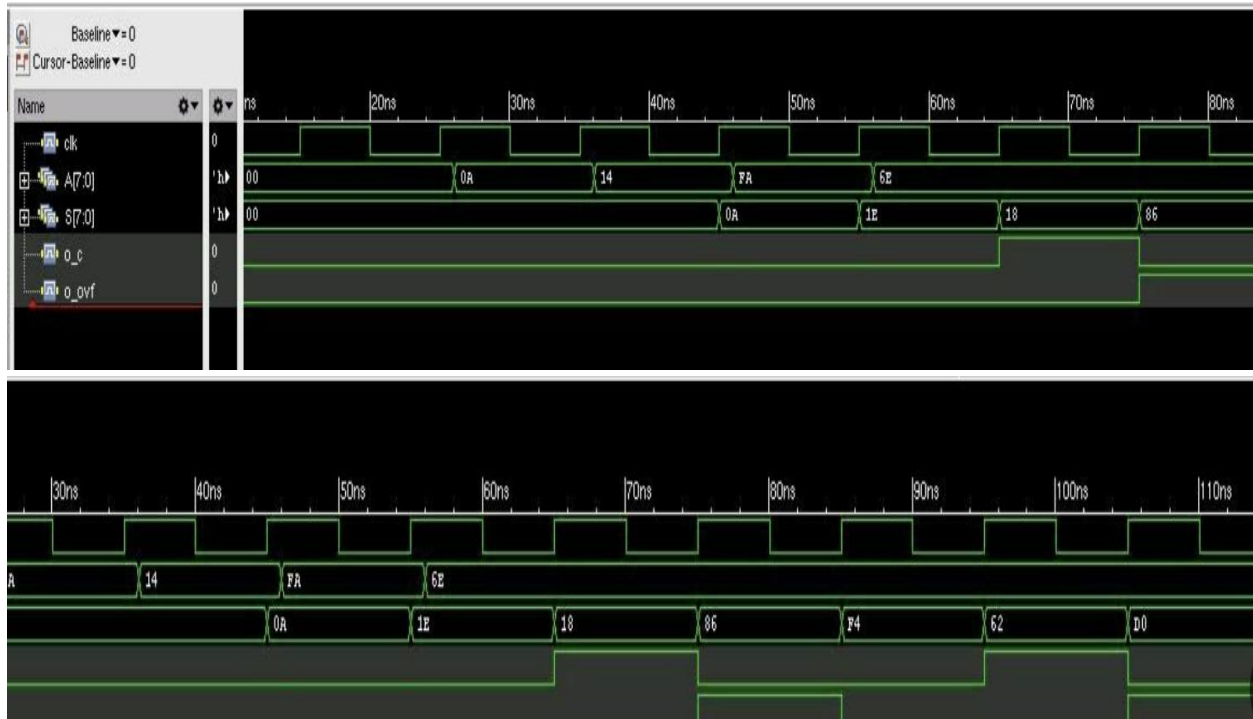
    A = 8'd110;
    @(posedge clk); #1; // S = 134, o_ovf = 1

    // --- 5. Kết thúc mô phỏng ---
    #50;
    $finish;
end

endmodule

```

Kiểm tra dạng sóng trên server test:



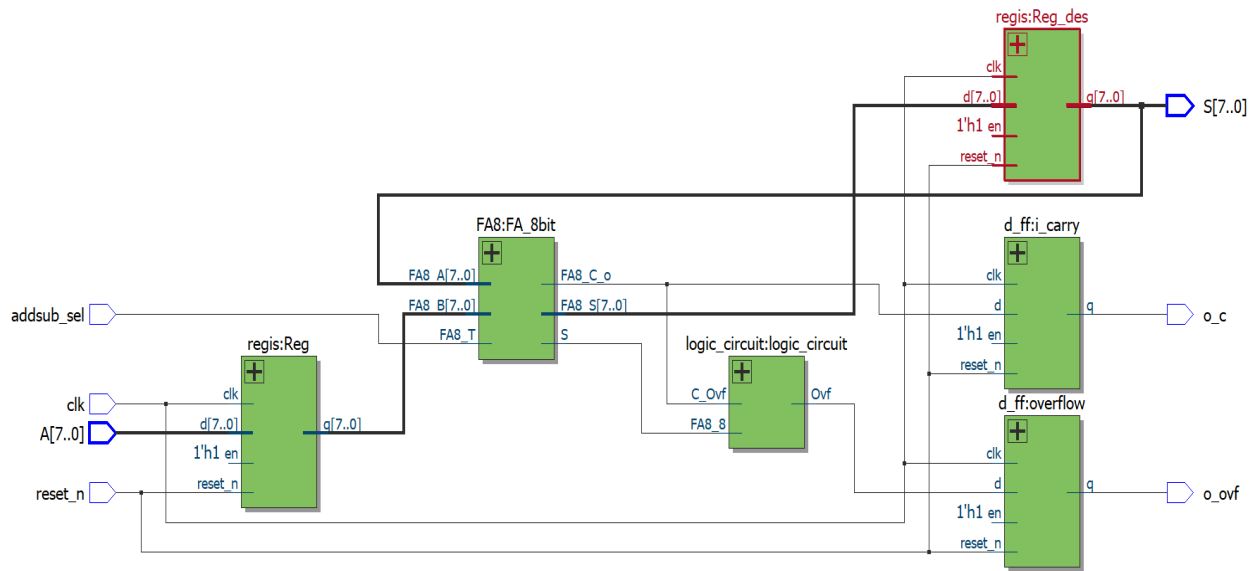
THÍ NGHIỆM 2

Mục tiêu: Xây dựng một hệ thống có khả năng thực hiện phép cộng hoặc trừ giá trị đầu vào A với chính nó.

Yêu cầu: Mở rộng mạch từ Thí nghiệm 1 để hỗ trợ cả hai chức năng: cộng và trừ. Thêm tín hiệu điều khiển add_sub vào mạch. Khi add_sub = 0: hệ thống sẽ cộng giá trị A vào thanh ghi S (giống như trong Thí nghiệm 1). Khi add_sub = 1: hệ thống sẽ trừ giá trị A khỏi thanh ghi S.

- Thiết kế sơ đồ mạch logic thể hiện đầy đủ các khối chức năng và kết nối.
- Viết mã mô tả phân cứng (HDL) bằng SystemVerilog để hiện thực hóa mạch này.
- Xây dựng testbench để mô phỏng và kiểm tra hoạt động của mạch, bao gồm đầy đủ các trường hợp điển hình và các trường hợp đặc biệt (như tràn số, số âm, giá trị biên).

Vẽ sơ đồ mạch:



Ý tưởng code:

Module chính bao gồm các module con là một Full adder 8 bit được ghép nối từ những bộ Full adder con, module con thanh ghi được hợp thành bởi 8 D flipflop để lưu trữ giá trị 8 bit, bộ logic circuit để xử lý cờ N.

Code system Verilog:

- **Module chính:**

```
module Bai2_lab1(
    input logic [7:0] A,
    input logic clk, addsub_sel,
    input logic reset_n,
    output logic o_ovf, o_c,
    output logic [7:0] S
);

logic [7:0] FA_o;
logic FA_co;
logic [7:0] i_reg;
logic i_clk;
```

```

logic i_rd;
logic ovf;
logic to_ovf_d;
assign i_clk = clk;

//====THANH GHI CHUA GIA TRI DAU VAO====
regis Reg(
    .d (A),
    .reset_n(reset_n),
    .en   (1'b1),
    .q (i_reg),
    .clk (i_clk));

//==== FA8BIT =====
FA8 FA_8bit(
    .FA8_A (S),
    .FA8_B (i_reg),
    .FA8_T (addsub_sel),
    .FA8_S (FA_o),
    .FA8_C_o(FA_co),
    .S(ovf)
);

//====THANH GHI DICH = S +A=====
regis Reg_des
(
    .d (FA_o),
    .reset_n(reset_n),
    .en   (1'b1),

```

```

        .q (S),
        .clk (i_clk));

//===== KHOI XU TIN HIEU DAU RA=====
logic_circuit logic_circuit(
    .C_Ovf (FA_co),
    .FA8_8 (ovf),
    .Ovf(to_ovf_d)
); //XU LI TIN HIEU

d_ff overflow(
    .d(to_ovf_d),
    .reset_n(reset_n),
    .en    (1'b1),
    .q (o_ovf),
    .clk (i_clk)
); //OVERFLOW

d_ff i_carry(
    .d (FA_co),
    .reset_n(reset_n),
    .en    (1'b1),
    .q (o_c),
    .clk (i_clk)
); // CO CARRY
endmodule

```

- Module FA:

```

module fa(

```

```

    input logic FA_a, FA_b, C_i,
        output logic FA_S, C_o
);
    assign FA_S = FA_a ^ FA_b ^ C_i;
    assign C_o = (FA_a & FA_b) | (FA_b & C_i) | (FA_a & C_i);
endmodule

```

- Module FA 8 bit (Rippen full adder carry:

```

module FA8 (
    input logic [7:0] FA8_A, FA8_B,
    input logic      FA8_T,
    output logic [7:0] FA8_S,
    output logic      FA8_C_o, S
);
    wire [7:0] Bx = FA8_B ^ {8{FA8_T}};
    wire [8:0] c;
    assign c[0] = FA8_T;

    fa fa0 (.FA_a(FA8_A[0]), .FA_b(Bx[0]), .C_i(c[0]), .FA_S(FA8_S[0]), .C_o(c[1]));
    fa fa1 (.FA_a(FA8_A[1]), .FA_b(Bx[1]), .C_i(c[1]), .FA_S(FA8_S[1]), .C_o(c[2]));
    fa fa2
(.FA_a(FA8_A[2]), .FA_b(Bx[2]), .C_i(c[2]), .FA_S(FA8_S[2]), .C_o(c[3]));
    fa fa3
(.FA_a(FA8_A[3]), .FA_b(Bx[3]), .C_i(c[3]), .FA_S(FA8_S[3]), .C_o(c[4]));
    fa fa4
(.FA_a(FA8_A[4]), .FA_b(Bx[4]), .C_i(c[4]), .FA_S(FA8_S[4]), .C_o(c[5]));
    fa fa5
(.FA_a(FA8_A[5]), .FA_b(Bx[5]), .C_i(c[5]), .FA_S(FA8_S[5]), .C_o(c[6]));
    fa fa6
(.FA_a(FA8_A[6]), .FA_b(Bx[6]), .C_i(c[6]), .FA_S(FA8_S[6]), .C_o(c[7]));
    fa fa7 (.FA_a(FA8_A[7]), .FA_b(Bx[7]), .C_i(c[7]), .FA_S(FA8_S[7]), .C_o(c[8]));

    assign FA8_C_o = c[8];
    assign S = c[7];
endmodule

```

- Module thanh ghi:

```

module regis (
    input logic    clk,
    input logic    reset_n,
    input logic    en,
    input logic [7:0] d,
    output logic [7:0] q
);
    d_ff b0 (clk, reset_n, en, d[0], q[0]);
    d_ff b1 (clk, reset_n, en, d[1], q[1]);
    d_ff b2 (clk, reset_n, en, d[2], q[2]);
    d_ff b3 (clk, reset_n, en, d[3], q[3]);
    d_ff b4 (clk, reset_n, en, d[4], q[4]);
    d_ff b5 (clk, reset_n, en, d[5], q[5]);
    d_ff b6 (clk, reset_n, en, d[6], q[6]);
    d_ff b7 (clk, reset_n, en, d[7], q[7]);
endmodule

```

- Module D-flioflop:

```

module d_ff (
    input logic clk,
    input logic reset_n,
    input logic en,
    input logic d,
    output logic q
);
    always_ff @(posedge clk or negedge reset_n) begin
        if (!reset_n) q <= 1'b0;
        else if (en) q <= d;
    end
endmodule

```

- Module xử lí cờ Carry và cờ Overflow:

```

module logic_circuit(
    input C_Ovf, FA8_8,
    output logic Ovf
);

```

```
assign Ovf = C_Ovf ^ FA8_8;
endmodule
```

Testbench:

```
module tb_Bai2_lab1;
    logic    clk;
    logic    addsub_sel;
    logic [7:0] A;
    logic    reset_n;
    logic [7:0] S;
    wire     o_ovf;
    wire     o_c;

    Bai2_lab1 dut (
        .A      (A),
        .clk     (clk),
        .addsub_sel (addsub_sel),
        .o_ovf   (o_ovf),
        .o_c     (o_c),
        .S       (S),
        .reset_n (reset_n)
    );

    //===== SET CLOCK 100MHZ GIA DINH=====
    initial clk = 1'b0;
    always #5 clk = ~clk;

    initial begin
        A      = 8'h00;
        addsub_sel = 1'b0;
        reset_n  = 1'b0;      // CLEAR MOI THANH GHI
        repeat (3) @(posedge clk);
        reset_n  = 1'b1;
    end

    task automatic apply_vec(input logic [7:0] a_i, input logic sel_i);
```

```

begin
    @(negedge clk);
    A      = a_i;
    addsub_sel = sel_i;
    @(posedge clk);
    #1;
    $display("[%0t ns] A=%02h sel=%0b => S=%02h o_c=%0b o_ovf=%0b",
        $time, A, addsub_sel, S, o_c, o_ovf);
end
endtask

integer i;
initial begin

    @(posedge reset_n);
    repeat (2) @(posedge clk);

    // ADD
    apply_vec(8'h00, 1'b0);
    apply_vec(8'h01, 1'b0);
    apply_vec(8'h7F, 1'b0);
    apply_vec(8'h80, 1'b0);
    apply_vec(8'hFF, 1'b0);

    // SUB
    apply_vec(8'h00, 1'b1);
    apply_vec(8'h01, 1'b1);
    apply_vec(8'h7F, 1'b1);
    apply_vec(8'h80, 1'b1);
    apply_vec(8'hFF, 1'b1);

    // Random
    for (i = 0; i < 20; i++) begin
        apply_vec($urandom_range(0,255), $urandom_range(0,1));
    end

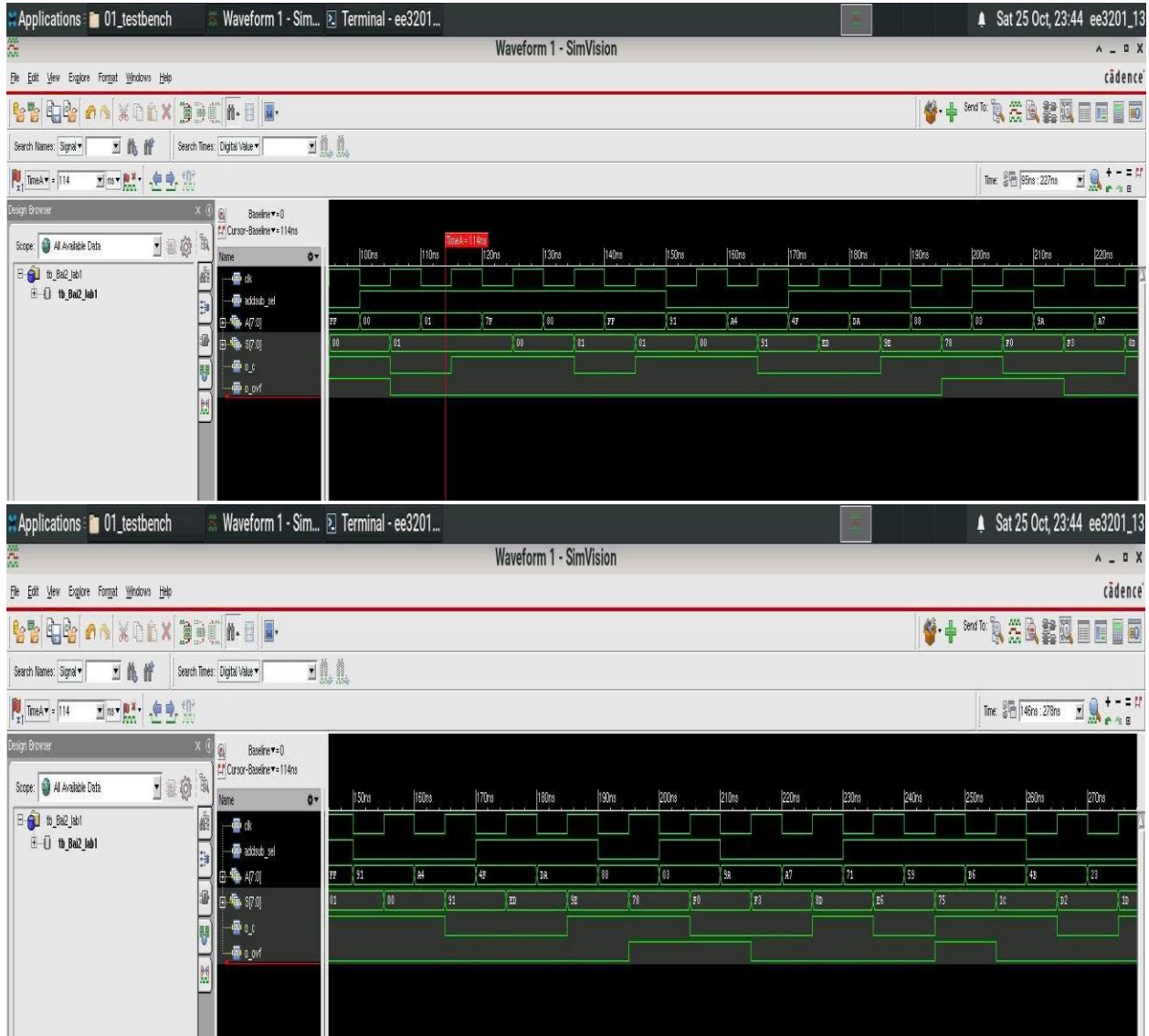
    $display("Testbench finished at %0t ns", $time);

```



```
#10 $finish;  
end  
endmodule
```

Kiểm tra dạng sóng trên server:

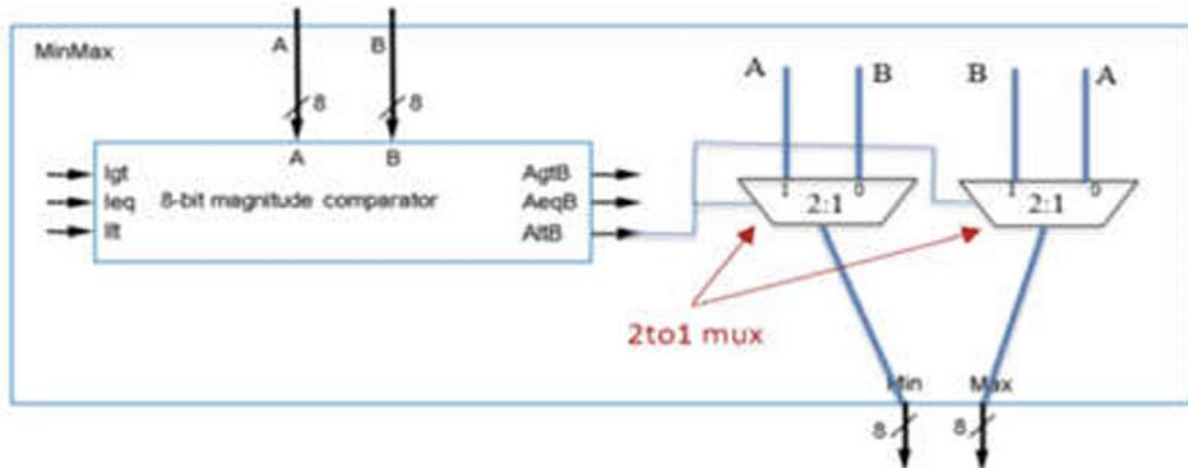


THÍ NGHIỆM 3

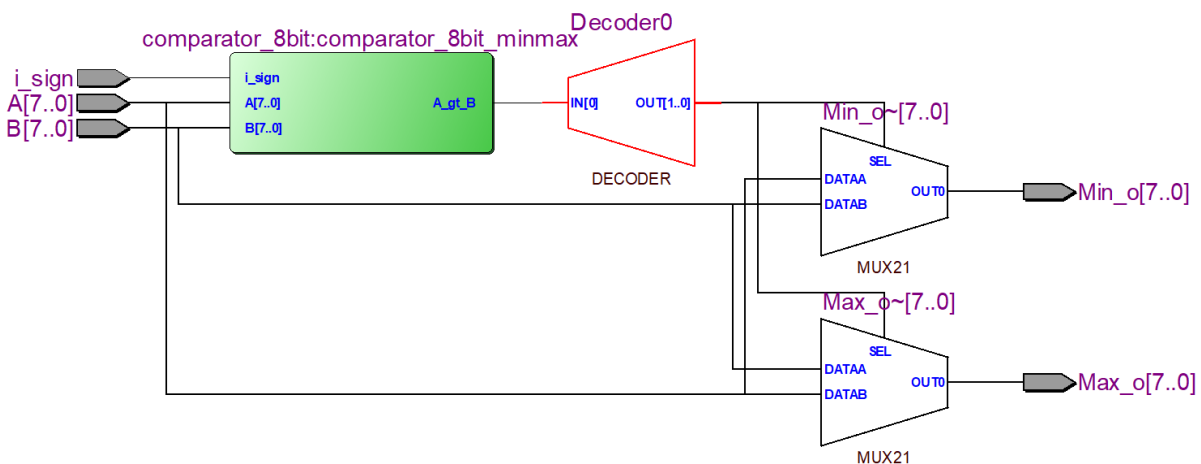
Mục tiêu: Biết cách thiết kế một mạch có chức năng xác định giá trị lớn hơn và nhỏ hơn giữa hai số 8-bit.

Mô tả: Mạch trong Hình 3 là một ví dụ về thiết kế dùng để xác định giá trị lớn hơn và nhỏ hơn giữa hai số 8-bit. Mạch này được xây dựng bằng một bộ so sánh (comparator) và

hai bộ chọn đa kênh (multiplexer). Bộ so sánh tạo ra tín hiệu điều khiển (select signal) cho đầu ra của các multiplexer.



Vẽ sơ đồ mạch:



Ý tưởng code:

Đầu tiên ta sẽ xây dựng một bộ so sánh 8-bit có tên là comparator_8bit. Module này có cấu trúc phân cấp: Bắt đầu với khối so sánh 2 bit không dấu trước, sau đó tạo khối so sánh 4 bit rồi đến khối so sánh 8 bit. Tiếp theo ta sẽ xây dựng module chính để tìm giá trị Min, Max dựa trên bộ so sánh 8 bit trên.

Code system Verilog:

- Module so sánh 2 bit

```
// 2-bits Comparator: Đây là khối cơ bản nhất, so sánh 2 số 2-bit.
module Comparator_2bits (
```

```

input logic [1:0] A_i,
input logic [1:0] B_i,
output logic A_lt_B,           // A nhỏ hơn B //
output logic A_gt_B,           // A lớn hơn B //
output logic A_eq_B            // A bằng B //
);
logic [1:0] A_xnor_B;
assign A_xnor_B = ~( A_i ^ B_i );
assign A_lt_B = ( ~ A_i [1] ) & B_i [1] | ( ~ A_i [1] ) & ( ~ A_i [0] ) & B_i [0] | ( ~
A_i [0] ) & B_i [1] & B_i [0];

assign A_eq_B = A_xnor_B [1] & A_xnor_B [0];

assign A_gt_B = ~(A_lt_B | A_eq_B);

endmodule : Comparator_2bits

```

- Module so sánh 4 bit

```

//4-bits Comparator: dùng 2 khối Comparator_2bits để xây dựng một khối 4-bit.
// ta chia ra làm hai: so sánh 2 bit cao: A[3:2] và B[3:2] và so sánh 2 bit thấp:
A[1:0] và B[1:0] rồi kết hợp kết quả lại
module Comparator_4bits (
input logic [3:0] A_i,
input logic [3:0] B_i,
output logic A_lt_B,
output logic A_gt_B,
output logic A_eq_B
);
logic [1:0] equal_o;           // Equal output

logic [1:0] less_o;           // Less than output

logic [1:0] greater_o;
// Logic so sánh nhỏ hơn
assign A_lt_B = less_o [1] | ( equal_o [1] & less_o [0] );
// Logic so sánh bằng: A = B nếu 2 bit cao và 2 bit thấp của cả 2 bằng nhau
assign A_eq_B = equal_o [1] & equal_o [0];
// Logic so sánh lớn hơn
assign A_gt_B = greater_o[1] | (equal_o[1] & greater_o[0]);

// tạo khối so sánh dùng Comparator_2bits để so sánh 2 bit cao

```

```

Comparator_2bits Comparator_2bits1 (
    .A_i( A_i[3:2] ),
    .B_i( B_i[3:2] ),
    .A_lt_B( less_o [1] ),
    .A_gt_B( greater_o[1] ),
    .A_eq_B( equal_o [1] )
);

// tạo khối so sánh dùng Comparator_2bits để so sánh 2 bit thấp
Comparator_2bits Comparator_2bits0 (
    .A_i( A_i[1:0] ),
    .B_i( B_i[1:0] ),
    .A_lt_B ( less_o [0] ),
    .A_gt_B( greater_o[0] ),
    .A_eq_B ( equal_o [0] )
);

endmodule : Comparator_4bits

```

- Module so sánh 8 bit

```

module comparator_8bit (
    input logic [7:0] A,
    input logic [7:0] B,
    input logic i_sign,           //1 là so sánh có dấu, 0 là so sánh không dấu
    output logic A_lt_B,         // ngõ ra A < B
    output logic A_gt_B,         // ngõ ra A > B
    output logic A_eq_B          // ngõ ra A = B
);

    logic [1:0] equal_o;          // ngõ ra bằng của bộ so sánh 4 bit
    logic [1:0] less_o;          // ngõ ra bé hơn của bộ so sánh 4 bit
    logic [1:0] greater_o;

    // Bắt đầu so sánh bằng
    assign A_eq_B = equal_o[1] & equal_o[0];

    // bắt đầu so sánh bé hơn
    logic unsigned_less_than;    // ngõ ra so sánh theo kiểu không dấu
    logic signed_less_than;      // ngõ ra so sánh theo kiểu có dấu

```

// Kết quả so sánh không dấu

```
assign unsigned_less_than = less_o[1] | (equal_o[1] & less_o[0]);
```

// Kết quả so sánh có dấu

```
assign signed_less_than = (A[7] & ~B[7]) | (~A[7] ^ B[7]) & unsigned_less_than;
```

// bắt đầu so sánh lớn hơn

```
logic unsigned_greater_than; // ngõ ra so sánh theo kiểu không dấu
```

```
logic signed_greater_than; // ngõ ra so sánh theo kiểu có dấu
```

// $A > B$ nghĩa là 2 điều kiện $A = B$ và $A < B$ đều sai.

```
assign unsigned_greater_than = ~(unsigned_less_than | A_eq_B);
```

```
assign signed_greater_than = ~(signed_less_than | A_eq_B);
```

```
always_comb begin // 3-1 Mux //
```

```
if ( i_sign ) begin
```

```
    A_lt_B = signed_less_than;
```

```
    A_gt_B = signed_greater_than;
```

```
end
```

```
else begin
```

```
    A_lt_B = unsigned_less_than;
```

```
    A_gt_B = unsigned_greater_than;
```

```
end
```

```
end
```

// tạo khối so sánh 8 bit

```
Comparator_4bits Comparator_4bits1 ( // so sánh 4 bit cao
```

```
    .A_i ( A[7:4] ),
```

```
    .B_i ( B[7:4] ),
```

```
    .A_lt_B ( less_o[1] ),
```

```
    .A_gt_B ( greater_o[1] ),
```

```
    .A_eq_B ( equal_o[1] )
```

```
);
```

```
Comparator_4bits Comparator_4bits0 ( // so sánh 4 bit thấp
```

```
    .A_i ( A[3:0] ),
```

```
    .B_i ( B[3:0] ),
```

```
    .A_lt_B ( less_o[0] ),
```

```
    .A_gt_B ( greater_o[0] ),
```

```
    .A_eq_B ( equal_o[0] )
```

```
);
```

```
endmodule: comparator_8bit
```

- Module tìm Min Max:

```
module MinMax (  
    input logic [7:0] A,  
    input logic [7:0] B,  
    input logic i_sign,          // tín hiệu lựa chọn: 1 là có dấu, 0 là không dấu  
    output logic [7:0] Min_o,  
    output logic [7:0] Max_o  
);  
  
    logic AgtB;                  // ngõ ra nội A lớn hơn B  
    logic AeqB;  
    logic AltB;  
  
    // kết nối với bộ so sánh 8 bit  
    comparator_8bit comparator_8bit_minmax (  
        .A ( A ),  
        .B ( B ),  
        .i_sign (i_sign),  
        .A_lt_B ( AltB ),  
        .A_gt_B (  AgtB ),  
        .A_eq_B ( AeqB )  
    );  
  
    // MUX 1: Dành cho ngõ ra Min_o  
    always_comb begin  
        case (AgtB)  
            1'b1: Min_o = B;      // Nếu AgtB=1 (A > B), Min = B  
            default: Min_o = A;   // Ngược lại (A <= B), Min là A  
        endcase  
    end  
  
    // MUX 2: Dành cho ngõ ra Max_o  
    always_comb begin  
        case (AgtB)  
            1'b1: Max_o = A;      // Nếu AgtB=1 (A > B), Max là A  
            default: Max_o = B;   // Ngược lại (A <= B), Max là B  
        endcase  
    end
```

```
endmodule: MinMax
```

Mô tả hoạt động của khối MinMax:

Đây là một bộ tìm giá trị Min và giá trị Max 8-bit bằng cách so sánh (có dấu hoặc không dấu) 2 dữ liệu đầu vào 8-bit A và B. Khối MinMax này được xây dựng từ:

- 1 bộ so sánh 8-bit comparator_8bit được thiết kế theo kiểu phân cấp (Hierarchical Design) từ một bộ so sánh 2-bit comparator_2bits. Bộ so sánh 8-bit này có 3 ngõ vào là dữ liệu 8-bit A, B và bit i_sign (dùng để quyết định so sánh có dấu hay không dấu).
- 2 bộ MUX 2-1 để xuất giá trị Min và giá trị Max. Đầu vào của 2 bộ MUX này là giá trị A và B.

Ngõ vào: dữ liệu 8-bit A, B và một tín hiệu 1-bit i_sign được đưa vào mạch

Bắt đầu so sánh:

- A và B được đưa vào khối comparator_8bit. Khối này sẽ nhìn vào bit i_sign để quyết định xem phải so sánh A và B theo kiểu có dấu hay không dấu.
- Sau khi so sánh xong, khối comparator_8bit sẽ báo kết quả tại 3 ngõ ra nội bộ (A_gt_B, A_lt_B, A_eq_B). Ví dụ $A < B$ nên $A_lt_B = 1$

Lựa chọn và xuất ra:

- Trong khối MinMax này, chúng ta chỉ cần sử dụng duy nhất 1 ngõ ra nội bộ là A_gt_B bởi vì: Nếu $A_gt_B = 1$ nghĩa là $A > B$, xuất giá trị Min = B và Max = A, nếu $A_gt_B = 0$ có nghĩa là $A \leq B$, xuất giá trị Min = A và Max = B
- Lưu ý là tại trường hợp $A = B$, lúc này giá trị Min = Max nên ta xuất giá trị Min = Max = A hay B đều được.
- Ngõ ra nội bộ A_gt_B sẽ được dùng làm tín hiệu lựa chọn cho 2 bộ MUX 2to1, 1 bộ MUX dùng để tìm giá trị Min, bộ còn lại dùng để tìm giá trị Max
- Cùng lúc đó, hai giá trị A và B ban đầu được đưa đến sẵn ở đầu vào của hai bộ MUX
- Bộ MUX tìm Min: Sau khi nhận được tín hiệu $A_gt_B = 1$ nghĩa là $A > B$, nó sẽ chọn B để đưa tới ngõ ra Min_o.
- Bộ MUX tìm Max: Sau khi nhận được tín hiệu $A_gt_B = 1$ nghĩa là $A > B$, nó sẽ chọn A để đưa tới ngõ ra Max_o.

Một vài lưu ý:

- Khối logic làm nhiệm vụ giải mã tín hiệu AgtB để tạo ra tín hiệu SEL cuối cùng là Decoder. Về cơ bản, trong sơ đồ RTL này, khối Decoder0 chỉ là một cách biểu diễn của logic trung gian có nhiệm vụ lấy kết quả so sánh và dùng nó làm tín hiệu lựa chọn.

Testbench:

```

module testbench_3;

  logic [7:0] A_tb;
  logic [7:0] B_tb;
  logic sign_tb;
  logic [7:0] Min_tb;
  logic [7:0] Max_tb;

  MinMax dut (
    .A ( A_tb ),
    .B ( B_tb ),
    .i_sign (sign_tb),
    .Min_o ( Min_tb ),
    .Max_o ( Max_tb )
  );

  initial begin
    $dumpfile("testbench_bai3.vcd");
    $dumpvars(0, testbench_3);

    // -----
    // Gán giá trị cho 2 ngõ vào A và B
    // TH1
  end

```



```

A_tb = 8'b11110000;
B_tb = 8'b11110000;
    sign_tb = 0;
#20;

A_tb = 8'b11110000;
B_tb = 8'b11110000;
    sign_tb = 0;
#20;

// TH2
A_tb = 8'b11111111;
B_tb = 8'b10100000;
    sign_tb = 0;

#20;

A_tb = 8'b11111111;
B_tb = 8'b10100000;
    sign_tb = 0;
#20;

// TH3
A_tb = 8'b11110000; //240
B_tb = 8'b01111100; //
    sign_tb = 0;

#20;

```

```
A_tb = 8'b11110000;
B_tb = 8'b01111100;
    sign_tb = 0;
#20;

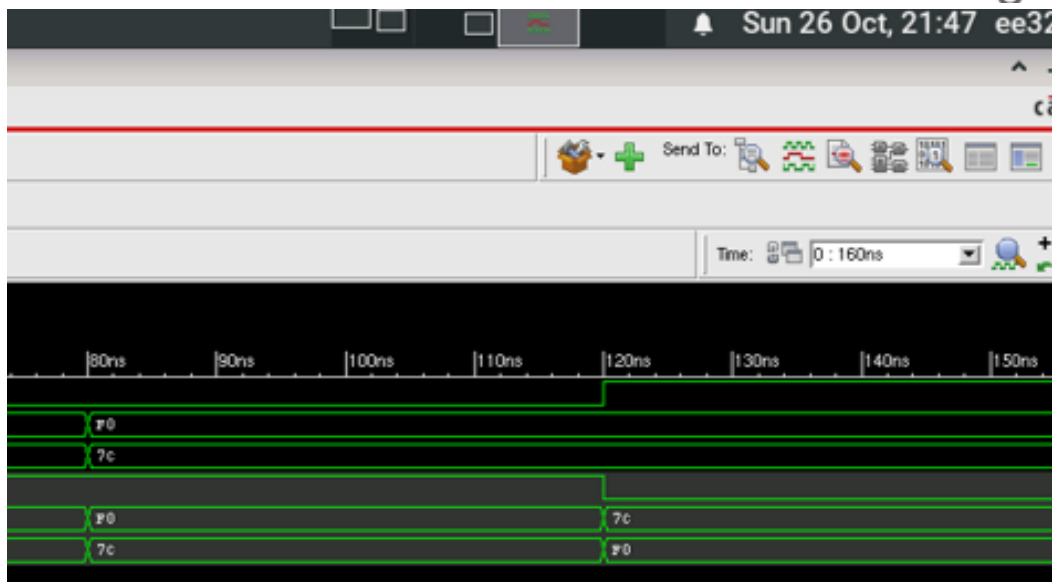
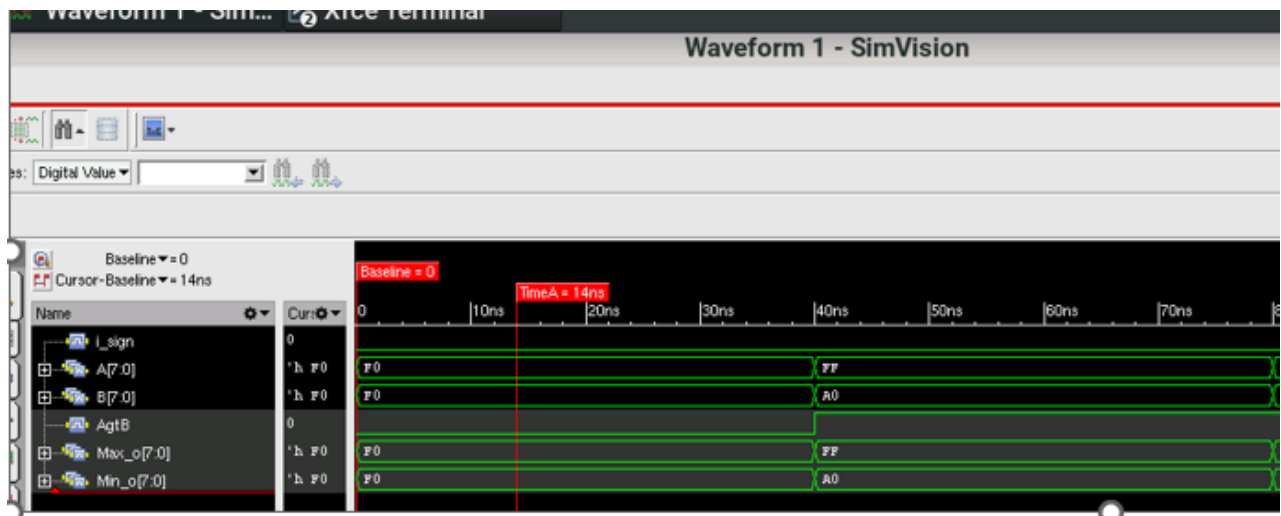
    // TH4
A_tb = 8'b11110000;
B_tb = 8'b01111100;
    sign_tb = 1;

#20;
A_tb = 8'b11110000;
B_tb = 8'b01111100;
    sign_tb = 1;
#20;

$finish;

end
endmodule
```

Kiểm tra dạng sóng trên server test:



- Trường hợp 1: $i_sign = 0$, $A = B = 0xF0 \rightarrow \text{Max} = \text{Min} = A = B = F0$
- Trường hợp 2: $i_sign = 0$, $A = 0xFF = 255$, $B = 0xA0 = 160 \rightarrow A > B \rightarrow \text{Max} = A = FF$ và $\text{Min} = B = A0$
- Trường hợp 3: $i_sign = 0$, $A = 0xF0 = 240$, $B = 0x7c = 124 \rightarrow A > B \rightarrow \text{Max} = A = 0xF0$, $\text{Min} = B = 0xA0$
- Trường hợp 4: $i_sign = 1$, $A = 0xF0 = -16$, $B = 0x7c = 124 \rightarrow B > A \rightarrow \text{Max} = B = 0x7C$, $\text{Min} = A = 0xF0$

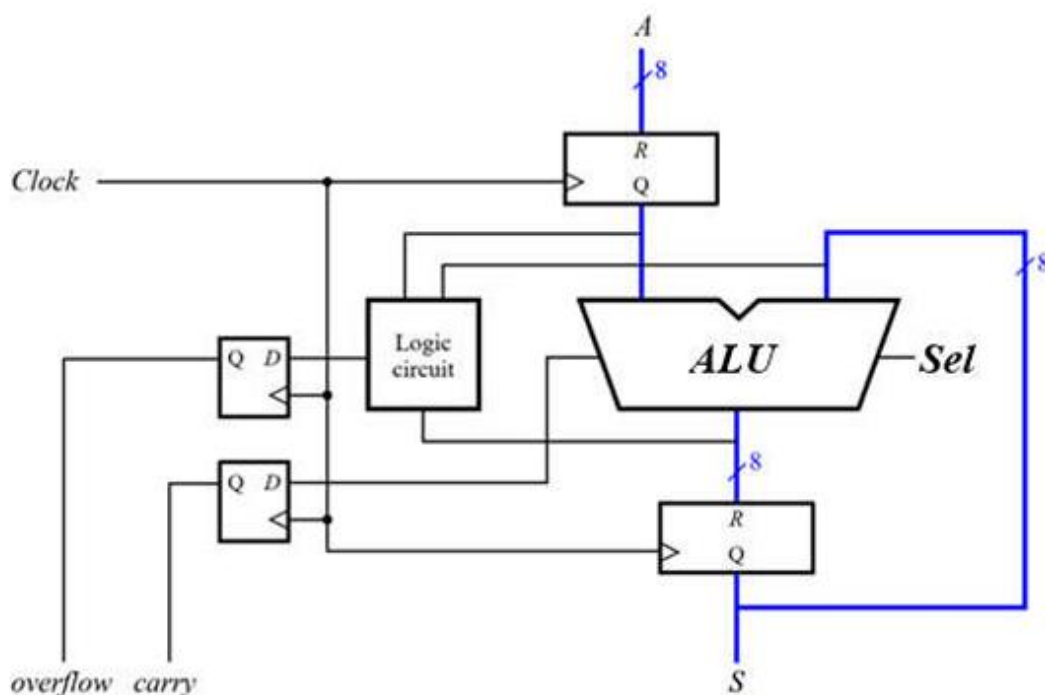
THÍ NGHIỆM 4

Mục tiêu:

Mạch ALU có hai ngõ vào A, B và một ngõ điều khiển Sel. ALU thực hiện 4 phép toán tùy theo giá trị Sel:

Sel	Phép toán
00	Cộng (Add)
01	Trừ (Subtract)
10	Lấy giá trị lớn nhất (Maximum)
11	Lấy giá trị nhỏ nhất (Minimum)

Các giá trị A, B, và kết quả S đều được lưu vào thanh ghi (cập nhật theo xung clock).
Có thể có tín hiệu carry hoặc overflow được xuất ra.



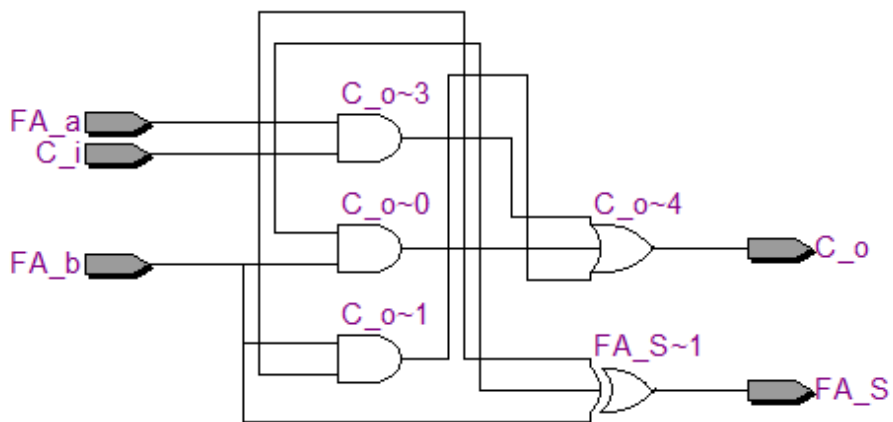
Ý tưởng code:

Ta sẽ thiết kế các bộ bên ngoài trước bao gồm 2 bộ D_Flip flop và 2 bộ Reg, bộ ALU sẽ bao gồm những khối là bộ MinMax, bộ cộng, trừ và các cổng MUX dùng để lựa chọn theo bảng yêu cầu và 1 bộ logic circuit dùng để xét overflow và carry. Sau khi nối các dây lại, bộ hoàn chỉnh sẽ trả về kết quả S và 2 cờ carry và overflow, bộ hoạt động theo xung cạnh lên.

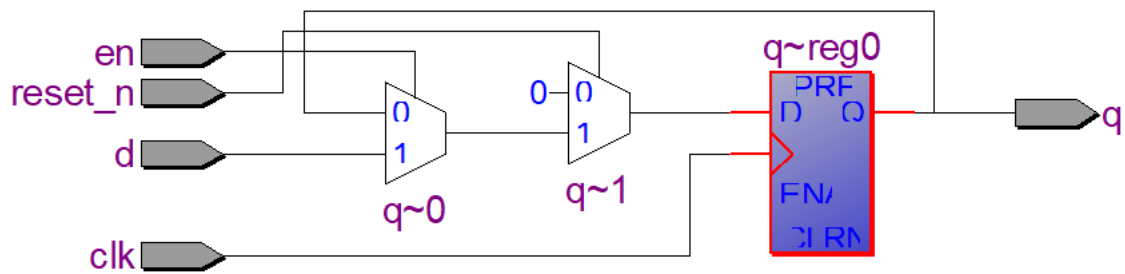
Vẽ sơ đồ mạch:

Các bộ được sử dụng trong ALU:

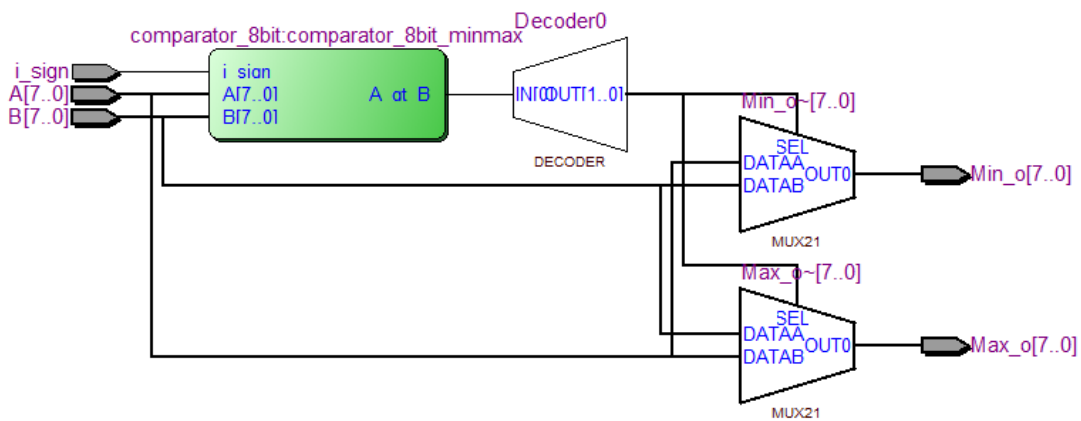
- Bộ cộng FULL ADDER



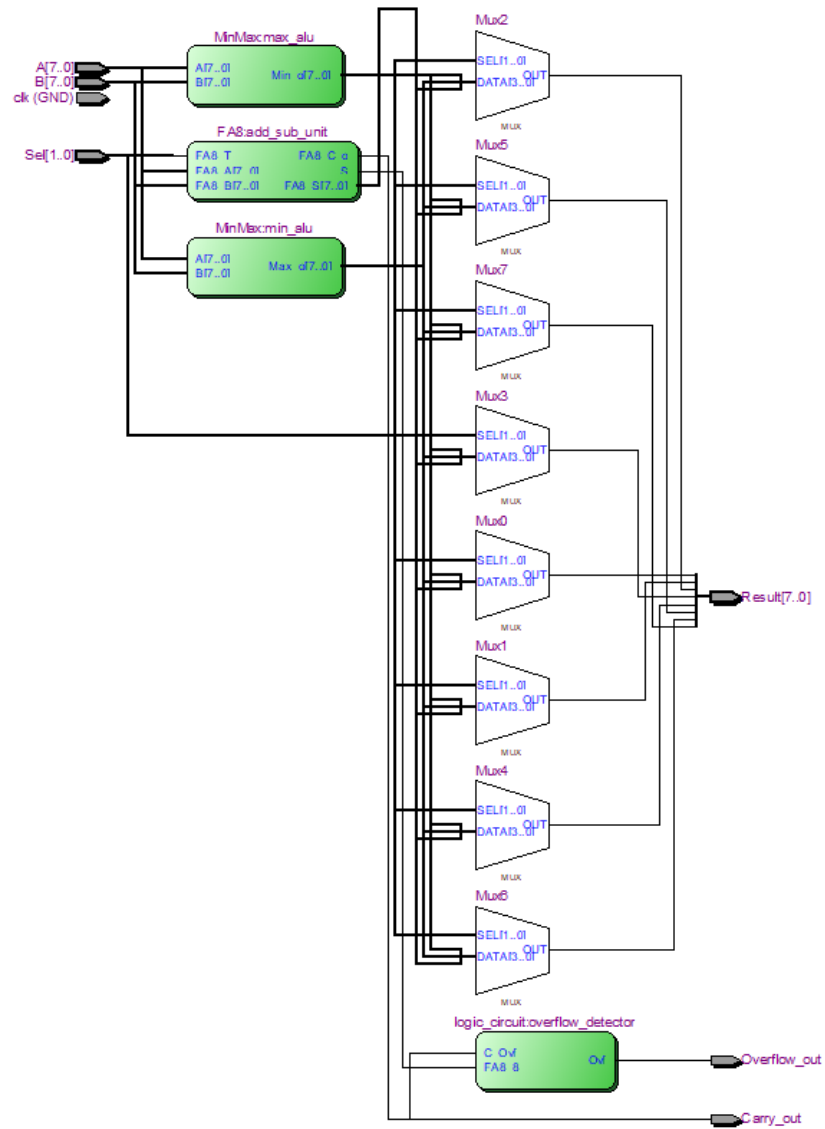
- Bộ D_Flip Flop



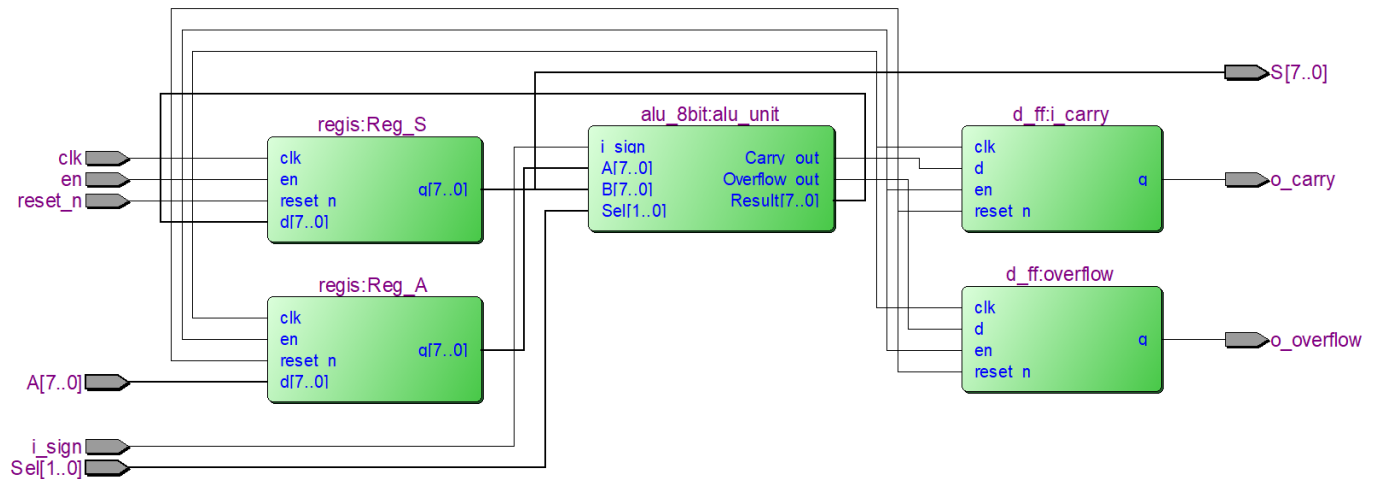
- Bộ xét Min Max



- Bộ tổng hợp ALU



- Bộ hoàn chỉnh theo đề



Code system Verilog:

Các bộ được sử dụng trong ALU:

- Bộ cộng FULL ADDER

```
module fa(
    input logic FA_a, FA_b, C_i,
    output logic FA_S, C_o
);
    assign FA_S = FA_a ^ FA_b ^ C_i;
    assign C_o = (FA_a & FA_b) | (FA_b & C_i) | (FA_a & C_i);
endmodule
```

- Bộ D_Flip Flop

```
module d_ff(
    input logic clk,
    input logic reset_n,
    input logic en,
    input logic d,
    output logic q
);
    always_ff @(posedge clk) begin
        if (!reset_n) q <= 1'b0;
        else if (en) q <= d;
    end
endmodule
```

- Bộ xét Min Max

```

/* phương pháp là xây dựng một bộ so sánh 8-bit có cấu trúc phân cấp
bắt đầu với khối so sánh 2 bit không dấu trước, sau đó tạo khối so sánh 4 bit
để so sánh 8 bit: chia ra làm hai nửa: so sánh 4 bit cao: A[7:4] với B[7:4] và so sánh
4 bit thấp: A[3:0] với B[3:0] rồi kết hợp kết quả lại
*/

module Comparator_2bits (
    input logic [1:0] A_i,
    input logic [1:0] B_i,
    output logic A_lt_B, // A nhỏ hơn B //
    output logic A_gt_B, // A lớn hơn B //
    output logic A_eq_B // A bằng B //
);
    logic [1:0] A_xnor_B;
    assign A_xnor_B = ~(A_i ^ B_i);
    assign A_lt_B = (~A_i[1] & B_i[1]) | (~A_i[0] & B_i[0]);
    assign A_eq_B = A_xnor_B[1] & A_xnor_B[0];
    assign A_gt_B = ~(A_lt_B | A_eq_B);

endmodule : Comparator_2bits

//
// 4-bits Comparator: dùng 2 khối Comparator_2bits để xây dựng một khối 4-bit.
// ta chia ra làm hai: so sánh 2 bit cao: A[3:2] và B[3:2] và so sánh 2 bit thấp:
// A[1:0] và B[1:0] rồi kết hợp kết quả lại
//
module Comparator_4bits (
    input logic [3:0] A_i,
    input logic [3:0] B_i,
    output logic A_lt_B, // A nhỏ hơn B //
    output logic A_gt_B, // A lớn hơn B //
    output logic A_eq_B // A bằng B //
);
    logic [1:0] equal_o; // Equal output
    logic [1:0] less_o; // Less than output
    logic [1:0] greater_o;

```



```

    assign A_lt_B = less_o [1] | ( equal_o [1] & less_o [0] );          /* Logic so
sánh nhỏ hơn: A < B nếu 2 bit cao của A < B HOẶC
                                     2 bit cao của A = B VÀ 2 bit thấp
của A < B */
    assign A_eq_B = equal_o [1] & equal_o [0];                        //
Logic so sánh bằng: A = B nếu 2 bit cao và 2 bit thấp của cả 2 bằng nhau
    assign      A_gt_B = greater_o[1] | (equal_o[1] & greater_o[0]); //A > B nếu 2
bit cao A > B, HOẶC 2 bit cao A=B và 2 bit thấp A > B

Comparator_2bits Comparator_2bits1 (                                // tạo khối so sánh
dùng Comparator_2bits tên là Comparator_2bits1 để so sánh 2 bit cao
    .A_i( A_i[3:2] ),
    .B_i( B_i[3:2] ),
    .A_lt_B( less_o [1] ),
    .A_gt_B( greater_o[1] ),
    .A_eq_B( equal_o [1] )
);
Comparator_2bits Comparator_2bits0 (                                // tạo khối so sánh
dùng Comparator_2bits tên là Comparator_2bits1 để so sánh 2 bit thấp
    .A_i( A_i[1:0] ),
    .B_i( B_i[1:0] ),
    .A_lt_B ( less_o [0] ),
    .A_gt_B( greater_o[0] ),
    .A_eq_B ( equal_o [0] )
);
endmodule : Comparator_4bits

// ----- bộ so sánh 8 bit -----
-----
module comparator_8bit (
    input logic [7:0] A,
    input logic [7:0] B,
    input logic i_sign,          // ngõ vào lựa chọn, 1 là so sánh có dấu, 0 là so sánh
không dấu
    output logic A_lt_B,          // ngõ ra A < B
    output logic A_gt_B,          // ngõ ra A > B
    output logic A_eq_B          // ngõ ra A = B
);

```

```

logic [1:0] equal_o; // ngõ ra bằng của bộ so sánh 4
bit: 4-bit comparator //
logic [1:0] less_o; // ngõ ra bé hơn của bộ so sánh 4
bit: 4-bit comparator //
logic [1:0] greater_o;

// Bắt đầu so sánh bằng
assign A_eq_B = equal_o[1] & equal_o[0];
// A = B nếu cả 4 bit cao và 4 bit thấp của A và B đều giống nhau

// bắt đầu so sánh bé hơn
logic unsigned_less_than; // ngõ ra so sánh theo kiểu không
dấu
logic signed_less_than; // // ngõ ra so sánh theo kiểu có dấu

// Kết quả so sánh không dấu
assign unsigned_less_than = less_o[1] | (equal_o[1] & less_o[0]); // A < B nếu 4
bit cao của A < B HOẶC 4 bit cao của A = B VÀ 4 bit thấp của A < B

// Kết quả so sánh có dấu
assign signed_less_than = (A[7] & ~B[7]) | (~A[7] ^ B[7]) &
unsigned_less_than);

// bắt đầu so sánh lớn hơn
logic unsigned_greater_than; // ngõ ra so sánh theo kiểu không
dấu
logic signed_greater_than; // // ngõ ra so sánh theo kiểu
có dấu

// A > B nghĩa là 2 điều kiện A = B và A < B đều sai.
assign unsigned_greater_than = ~(unsigned_less_than | A_eq_B); //
không dấu
assign signed_greater_than = ~(signed_less_than | A_eq_B);
// có dấu

always_comb begin // 3-1 Mux //
if ( i_sign ) begin // Nếu i_sign = 1, chọn KẾT QUẢ CÓ DẤU, i_sign = 0 thì
chọn kq không dấu
A_lt_B = signed_less_than;
A_gt_B = signed_greater_than;

```

```

        end
    else begin
        A_lt_B = unsigned_less_than;
        A_gt_B = unsigned_greater_than;
    end
end

Comparator_4bits Comparator_4bits1 (
    .A_i ( A[7:4] ),
    .B_i ( B[7:4] ),
    .A_lt_B ( less_o[1] ),
    .A_gt_B ( greater_o[1]),
    .A_eq_B ( equal_o[1] )
);
Comparator_4bits Comparator_4bits0 (
    .A_i ( A[3:0] ),
    .B_i ( B[3:0] ),
    .A_lt_B ( less_o[0] ),
    .A_gt_B ( greater_o[0]),
    .A_eq_B ( equal_o[0] )
);
endmodule: comparator_8bit

//-----bộ tìm Min Max-----
-----
module MinMax (
    input logic [7:0] A,
    input logic [7:0] B,
    input logic i_sign,          // ngõ vào lựa chọn, 1 là so sánh có dấu, 0 là so sánh
    không dấu
    output logic [7:0] Min_o,      // ngõ ra A < B
    output logic [7:0] Max_o
);

    logic AgtB;
    logic AeqB;
    logic AltB;

    comparator_8bit comparator_8bit_minmax (
        .A ( A ),
        .B ( B ),

```

```

.i_sign (i_sign),
.A_lt_B ( AltB ),
.A_gt_B ( AgtB ),
.A_eq_B ( AeqB )
);

// MUX 1: Dành cho ngõ ra Min_o
always_comb begin
    case (AgtB)
        1'b1: Min_o = B; // Nếu AgtB=1 ( $A > B$ ), ngõ ra Min là B
        default: Min_o = A; // Ngược lại ( $A \leq B$ ), ngõ ra Min là A
    endcase
end

// MUX 2: Dành cho ngõ ra Max_o

always_comb begin
    case (AgtB)
        1'b1: Max_o = A; // Nếu AgtB=1 ( $A > B$ ), ngõ ra Max là A
        default: Max_o = B; // Ngược lại ( $A \leq B$ ), ngõ ra Max là B
    endcase
end

endmodule: MinMax

```

- **Bộ tổng hợp ALU**

```

// module chính alu điều khiển tất cả các lệnh, là một khối lớn (ALU) được ghép từ
các khối nhỏ hơn đã có sẵn
module alu_8bit (
    input logic clk,
    input logic [7:0] A,
    input logic [7:0] B,
    input logic [1:0] Sel,
    output logic [7:0] Result,           // // Kết quả của ALU
    output logic Carry_out,
    output logic Overflow_out
);

    // Dây nối để nhận kết quả từ khối FA8
    logic [7:0] add_sub_result;

```

```

logic    final_carry_out;          // c[8] - Carry out cuối cùng
logic    msb_carry_in;             // c[7] - Carry in của bit 7

    // Dây nối để nhận kết quả từ khối logic_circuit
logic    out_overflow;             // Cờ tràn

    // 1. Instantiate bộ cộng/trừ 8-bit
    // Tận dụng Sel[0] để điều khiển FA8_T:
    // Khi Sel = 00 (Add), Sel[0] = 0.
    // Khi Sel = 01 (Sub), Sel[0] = 1.
    FA8 add_sub_unit (
        .FA8_A  (A),
        .FA8_B  (B),
        .FA8_T  (Sel[0]),
        .FA8_S  (add_sub_result),
        .FA8_C_o (final_carry_out),
        .S      (msb_carry_in)      // Output 'S' của FA8 là c[7]
    );

    // 2. Instantiate bộ tính toán cờ tràn
    logic_circuit overflow_detector (
        .C_Ovf (final_carry_out),    // Nối carry-out của FA8 vào
        .FA8_8 (msb_carry_in),      // Nối carry-in MSB của FA8 vào
        .Ovf   (out_overflow)
    );

    // Khai báo các dây nối max, min
    logic [7:0] max_result; // Dây nhận kết quả max
    logic [7:0] min_result; // Dây nhận kết quả min

    // Gọi (Instantiate) bộ tính toán min max
    MinMax min_alu ( .A( A ), .B( B ), .Max_o( max_result ) );

    MinMax max_alu ( .A( A ), .B( B ), .Min_o( min_result ) );

    // Dùng một bộ MUX để chọn kết quả cuối cùng đưa ra ngoài
    always_comb begin
        Result = 8'b0; // Mặc định ngõ ra = 0 để tránh tạo latch

        case (Sel)
            2'b00: begin

```

```

        Result = add_sub_result;                                // Nếu mã lệnh là
ADD, chọn kết quả từ bộ cộng
        Carry_out = final_carry_out;
        Overflow_out = out_overflow;
    end

    2'b01: begin
        Result = add_sub_result;
        Carry_out = final_carry_out;
        Overflow_out = out_overflow;
    end

    2'b10: begin
        Result = max_result;
        Carry_out = final_carry_out;
        Overflow_out = out_overflow;
    end

    2'b11: begin
        Result = min_result;
        Carry_out = final_carry_out;
        Overflow_out = out_overflow;
    end

    default: begin
        // Nếu mã lệnh không hợp lệ, ngõ ra = 0
        Result = 8'b0;
        Carry_out = 1'b0;
        Overflow_out = 1'b0 ;
    end
endcase
end

endmodule: alu_8bit

```

- Bộ hoàn chỉnh theo đề

```

module Bai4_lab1(
    input logic [7:0] A,
    input logic clk,
    input logic reset_n,
    input logic en,
    input logic [1:0] Sel,

```

```

        output logic o_overflow,          // Cờ tràn đã qua flip-flop
        output logic o_carry,            // Cờ nhớ đã qua flip-flop
        output logic [7:0] S             // Kết quả cuối cùng sau thanh ghi
    );

    // Dây chứa giá trị A sau khi qua thanh ghi đầu vào
    logic [7:0] A_reg;

    // Dây chứa kết quả và các cờ trực tiếp từ ALU (trước khi qua thanh ghi/FF)
    logic [7:0] alu_result;
    logic      alu_carry_out;
    logic      alu_overflow_out;

//Instantiate thanh ghi đầu vào cho A
regis Reg_A(
    .d(A),
    .q(A_reg),
    .clk (clk),
    .reset_n (reset_n),          // Nổi lên mức cao -> Không reset
    .en      (en)                // Nổi lên mức cao -> Luôn hoạt động
);

// Instantiate khối ALU chính
alu_8bit alu_unit (
    .A      (A_reg),
    .B      (S),                // Phản hồi từ đầu ra S
    .Sel     (Sel),
    .Result  (alu_result),
    .Carry_out (alu_carry_out),
    .Overflow_out (alu_overflow_out)
);

// Instantiate thanh ghi đầu ra cho S
regis Reg_S                                     // thanh ghi chứa data
S
(
    .d (alu_result),
    .q (S),
    .clk (clk),
    .reset_n (reset_n),          // Nổi lên mức cao -> Không reset
    .en      (en)                // Nổi lên mức cao -> Luôn hoạt động

```

```

    );
    // khối logic circuit để xử lý cờ tràn ovf đã có sẵn trong khối ALU rồi

    // Instantiate D-FlipFlop cho cờ Carry
    d_ff overflow(                                     // dff chứa cờ
    ocf
        .d(alu_overflow_out),                         //
        .q(o_overflow),                             // cờ ovf
        đã được lưu trong dff
        .clk(clk),
        .reset_n(reset_n), // Nổi lên mức cao -> Không reset
        .en(en)           // Nổi lên mức cao -> Luôn hoạt động
    );

    // Instantiate D-FlipFlop cho cờ Overflow
    d_ff i_carry(                                     // dff chứa cờ c
        .d(alu_carry_out),
        .q(o_carry),
        .clk(clk),
        .reset_n(reset_n), // Nổi lên mức cao -> Không reset
        .en(en)           // Nổi lên mức cao -> Luôn hoạt động
    );

endmodule

```

Testbench:

```

`timescale 1ns/1ps

module tb_B=bai1_lab1;

    // --- Khai báo tín hiệu ---

    logic [7:0] A;

    logic en;

    logic clk;

    logic reset_n;

    logic o_ovf;

    logic o_c;

```



```

logic [7:0] S;

// --- Instantiate DUT (Design Under Test) ---
Bai1_lab1 dut (
    .A      (A),
    .en     (en),
    .clk     (clk),
    .reset_n (reset_n),
    .o_ovf   (o_ovf),
    .o_c     (o_c),
    .S      (S)
);

// --- Tạo Clock ---
initial begin
    clk = 0;
    forever #5 clk = ~clk; // Chu kỳ 10ns
end

// --- Lệnh để tạo file dạng sóng ---
initial begin
    $dumpfile("waveform.vcd");
    $dumpvars(0, tb_Bai1_lab1);
end

// --- Kịch bản Test (Stimulus) ---
initial begin

```

```

// --- 1. Khởi tạo và Reset ---
A    = 8'd0;
en    = 1'b0;  // Tắt enable
reset_n = 1'b0;  // Kích hoạt Reset (mức thấp)
#20;          // Giữ reset trong 20ns
reset_n = 1'b1;  // Nhả Reset
en    = 1'b1;  // Bật Enable lên và giữ nguyên

@(posedge clk); #1; // Đợi 1 xung clock để S reset về 0

// --- 2. Test phép cộng đơn giản (en = 1) ---
A = 8'd10;
@(posedge clk); #1; // S = 0 + 10 = 10

A = 8'd20;
@(posedge clk); #1; // S = 10 + 20 = 30

// --- 3. Test CARRY (o_c = 1) (en = 1) ---
// S đang là 30. Ta + thêm A = 250.
A = 8'd250;
@(posedge clk); #1; // S = 24, o_c = 1

// --- 4. Test OVERFLOW (o_ovf = 1)
// Kích bản: S đang là 24 (số dương). Ta + thêm A = 110 (số dương).

A = 8'd110;
@(posedge clk); #1; // S = 134, o_ovf = 1

```

```
// --- 5. Kết thúc mô phỏng ---
#50;

$finish;

end

endmodule
```

Kiểm tra dạng sóng trên server:

